

Lesson 1 — What Each Notebook Shows

Notebook guide, lesson 1 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

2 October 2026 · three notebooks

Contents

Notebook 01 — 01_kinds_of_learning	1
Supervised, unsupervised, self-supervised	1
Notebook 02 — 02_first_ml_workflow	2
A complete machine learning workflow, start to finish	2
Notebook 03 — 03_how_models_mislead	3
Four ways a model looks excellent and is worthless	3

A short guide to this lesson's notebooks. For each one: what it is for, the concepts in the order they appear, the numbers worth noticing, and **the two-minute version**, a summary to read before running it or to come back to afterwards. Every number here comes from the committed output of the notebook it belongs to.

Notebook 01 — 01_kinds_of_learning

Supervised, unsupervised, self-supervised

What it is for. The three definitions are easy to recite and easy to misapply, so the notebook runs all three **on the same data** and the difference becomes something seen rather than memorised. The point it exists to make: the distinction is not about the algorithm, it is about **where the answer comes from**.

The data. 178 wines, 13 chemical measurements, three cultivars — a cultivar being a cultivated variety, here a grape variety.

The concepts, in order.

1. **Supervised — somebody recorded the answer.** Predicting the cultivar gives accuracy **0.981**, and the section points out what makes it the clean case: **the cultivar is not one of the measurements**, it is an answer written down about each wine. Regression is the same idea with a number for an answer; lesson 3 is about it.

2. **Unsupervised — nobody labelled anything.** k-means against the true cultivars gives an adjusted Rand index of **0.897**, unusually high because these chemical groups really are separated. Do not expect it of clustering in general.
3. **The `n_init=10` aside**, which is better than it looks: with a single random start, 20 seeds give **6 distinct solutions**, WCSS from 1277.9 to 1595.8 and ARI from **0.322 to 0.915** — worse than useless to nearly perfect, on identical data and an identical algorithm. Ten restarts collapse that to one solution.
4. **Self-supervised — part of the input becomes the answer.** Framed explicitly as an intuition, with the method left to a later course. Hide **flavanoids** — a real measurement nobody asked to predict — and recover it from the other twelve: $R^2 = 0.816$. The contrast with step 1 is the whole distinction: there the answer lived outside the measurements; here it is one of them.
5. **Reading R^2** , attached to that regression: recomputed by hand and against **r2_score** (both 0.816), and read as the share of the squared error of always guessing the mean that the model removed.
6. **Why anyone would do it, and where the notebook stops.** The prediction is a pretext; the point is what the model learns and can reuse. A linear regression learns nothing worth reusing, so that step is not shown.

The two-minute version. *One dataset, three questions. Predict the grape variety and it is supervised learning, 0.981 — and the grape variety is not a measurement, it is an answer somebody wrote down. Throw the labels away and cluster, and it is unsupervised, ARI 0.897, which is unusually lucky. Hide one of the measurements and predict it from the others, and it is self-supervised, R^2 0.816: nobody asked for flavanoids to be predicted, it is used as an answer because it was already there, which is why it costs nothing. That is only the intuition — the reason anyone does it, reusing what the model learned, belongs to a later course.*

Notebook 02 — 02_first_ml_workflow

A complete machine learning workflow, start to finish

What it is for. To walk the whole path once, on real data, so that you have a map before any individual method is studied. Nothing in it is optimised: the subject is the *shape* of the process and the decisions taken along the way.

The data. The Wisconsin breast cancer dataset — 569 tumours, 30 measurements each, 357 benign against 212 malignant.

The concepts, in order.

1. **Frame the problem before touching the data.** What is predicted, from what, and which error is harmful. A missed malignancy and a false alarm are not the same event.
2. **Load and look.** The class balance is the first number that matters: predicting “benign” for everyone is already right **62.7%** of the time.
3. **Split before anything else** — before scaling, before selection, before looking at correlations. 426 training rows against 143 test, with **stratify** keeping the balance (0.627 against 0.629).

4. **Baseline first.** `DummyClassifier` scores **0.629**, which is what gives every later number a meaning.
5. **A pipeline, not two steps.** `StandardScaler` learns a mean and a standard deviation, and inside a pipeline it learns them from the training rows only. Model accuracy **0.986**, an improvement of **+0.357** over the baseline.
6. **Accuracy is not the answer.** The classification report and the confusion matrix: two errors, and one of them is a malignant tumour called benign. With malignant as the positive class, $\text{recall} = \text{TP} / (\text{TP} + \text{FN})$ and $\text{precision} = \text{TP} / (\text{TP} + \text{FP})$: both **52/53 = 0.981**, equal only because there is one miss and one false alarm.
7. **The threshold nobody chose.** The model does not output labels: it outputs the probability that a tumour is malignant (`predict_proba`), and `predict` compares it with 0.5. Every number above was computed at that 0.5, which is a default and not a decision. The sweep is the most quotable table in the notebook:

threshold	missed	false alarms	recall	precision
0.10	0	11	1.000	0.828
0.50	1	1	0.981	0.981
0.90	7	0	0.868	1.000

The two-minute version. *This notebook is the workflow end to end. It goes question $\rightarrow$ data $\rightarrow$ split $\rightarrow$ baseline $\rightarrow$ pipeline $\rightarrow$ evaluation, in that order, and the order is the content. The model scores 0.986 against a baseline of 0.629 — and then we look at the two mistakes and find that one is a missed malignancy, which is why the last section moves the decision threshold and shows that the same model, unchanged, gives anything from “no malignancy missed with 11 false alarms” to “seven missed and no false alarms”. Same weights, different decision rule.*

Notebook 03 — 03_how_models_mislead

Four ways a model looks excellent and is worthless

What it is for. This is the notebook that stays with them. Each failure produces a number that would pass review, and **none of them is a coding error**.

The concepts, in order.

1. **Leakage.** 200 samples, 5000 pure-noise features, coin-flip labels — nothing to learn, so anything above 0.50 is an artefact. Select the best 20 features using all the data, then split: **0.767**. Do it correctly, inside the pipeline: **0.617**, against a truth of **0.500**.
2. **Why it works**, which is the part worth keeping: the kept columns have $|r|$ between 0.201 and 0.273 against a noise spread of 0.071 — **2.8 to 3.9 standard deviations from zero** — and the largest of 5000 draws is predicted at 0.292 by $\sigma\sqrt{2\ln n}$. The selector found the tails of a distribution centred on zero, and read them off all 200 rows, including the 60 about to become the test set.

3. **How far it goes.** Fewer rows hurt about four times as much as more columns: at 5000 columns fixed, 50 rows give **0.955**; at 200 rows fixed, going from 2,500 to 20,000 columns moves 0.776 to **0.830**.
4. **Imbalance.** 21 positives in 1500 rows. Always answering “negative” scores **0.986** with recall **0.000**; the trained model scores **0.987** with recall **0.048**, missing 20 of 21 positives.
5. **A shortcut feature.** Add `biopsy_scheduled`, a column recorded *because* the outcome was known: accuracy 0.986, and the model leans on it harder than on any real measurement (coefficient 2.131 against 0.780 for the next). The four deployment scenarios are the best part — **the two that raise an exception are the lucky ones**, because a silent 0.923 is a failure nobody notices.
6. **A single split is a noisy measurement.** 30 seeds: lowest **0.917**, highest **1.000**, mean 0.963, spread **0.083** from nothing but the choice of split. Cross-validation reports **0.960 ± 0.030**.

The two-minute version. *Four failures, none of them a bug. First, leakage: pure noise, coin-flip labels, and selecting features before splitting gives 77% where the truth is 50% — and it gets worse with fewer rows, four times faster than with more columns. Second, imbalance: 98.6% accuracy from a model that finds none of the 21 positives. Third, a shortcut column recorded because the answer was already known — and the alarming part is that the two deployment failures which raise an exception are the safe ones. Fourth, one split is one measurement: the same model scores anywhere from 0.917 to 1.000 depending on the seed. Every one of these notebooks runs cleanly and reports a plausible number.*